

Graph-Based Music Recommendation System

Gilberto Rios, Arul Pandita

Data Science, Applied Mathematics and Statistics, Johns Hopkins University, MD, USA

Course

EN.601.466 - Information Retrieval and Web Agents

Professor: David Yarowsky

Project Summary

We developed a hybrid music recommendation system that combines TF-IDF-based Information Retrieval techniques with graph-based similarity and a web scraping agent to improve recommendation quality and diversity.

May 2026

1. Introduction

Music recommendation systems have become an essential component of modern digital platforms, helping users discover new content in large and continuously growing music catalogs. Traditional recommendation approaches often rely on either content-based similarity or collaborative filtering; however, these methods can be limited in their ability to capture both semantic meaning and relational structure between items.

In this project, we propose MusicIR, a hybrid music recommendation system that integrates Information Retrieval (IR) techniques, graph-based modeling, and a web agent (scraper) to improve recommendation quality. The system leverages textual metadata scraped from real-world sources such as Wikipedia and Last.fm to build TF-IDF vector representations of artists, enabling semantic similarity through cosine similarity. In addition, a graph structure is constructed to capture relationships between artists based on shared genres and similarity patterns.

The core contribution of this project is the combination of these two perspectives: vector-space similarity and graph-based relationships, into a unified hybrid recommendation model. By introducing a weighted scoring mechanism, the system balances semantic and structural similarity, allowing for more diverse and contextually relevant recommendations.

To evaluate the effectiveness of the proposed approach, we compare a baseline TF-IDF model with the hybrid model using both qualitative examples and quantitative metrics such as Precision@K. The results demonstrate that incorporating graph-based information enhances the diversity and relevance of recommendations beyond what is achievable using traditional IR methods alone.

This project aligns with the goals of the course by combining key concepts from Information Retrieval, web agents, and graph-based modeling into a practical and interactive system.

2. How to Run the Project

Requirements

- Python 3.x .
- Required libraries: pandas, numpy, scikit-learn, networkx, streamlit, requests, beautifulsoup4.

Dependencies

`pip install pandas numpy scikit-learn networkx streamlit requests beautifulsoup4` .

Required Files (same directory as app.py)

- **dataset.csv**: base music dataset (around 20 MB) ([Spotify Tracks Dataset](#)).
- **artist_metadata_clean.csv**: scraped and processed artist bios.
- **baseline_similarity.npy**: cosine similarity matrix.
- **music_scraper.py**: web scraping agent (Wikipedia + Last.fm).

- **IR_and_WA_Project.ipynb**: full pipeline notebook (scrape → vectorize → graph → evaluate) .

Run the Application

streamlit run app.py

Usage

1. Enter an artist name (e.g., *Drake*, *ZAYN*).
2. Select number of recommendations (K).
3. Adjust α (balance between TF-IDF and graph).
4. Click button GET RECOMMENDATIONS.

The system displays baseline recommendations (TF-IDF only) and hybrid recommendations (TF-IDF + Graph).

Running the Full Pipeline (Notebook)

To re-scrape bios or recompute the similarity matrix, open **IR_and_WA_Project.ipynb** and run the cells in order:

1. **Cell 0** Web scraping agent: calls `music_scraper.run()`, which queries Wikipedia and Last.fm for the top artists in the dataset and saves `artist_metadata.csv` and `artist_metadata_clean.csv`. Automatically skipped if these files already exist.
2. **Cell 1** TF-IDF vectorization: builds the vector space model and saves `baseline_similarity.npy`.
3. **Cell 2** Graph construction: builds the music knowledge graph with genre and similarity edges.
4. **Cell 3** Evaluation: computes `Precision@5` for both models across all artists using genre-based ground truth derived automatically from `dataset.csv`.

3. System Overview

MusicIR is a hybrid recommendation system that integrates:

- Information Retrieval (TF-IDF + cosine similarity).
- Graph-based similarity (NetworkX).
- Web scraping (Wikipedia +Last.fm).

The system pipeline is:

1. Dataset Initialization (CSV).
2. Web Agent (Scraper).
3. Text Processing & Vectorization (TF-IDF).
4. Graph Construction.
5. Hybrid Recommendation Engine.

The dataset (dataset.csv) contains approximately 114,000 tracks across 114 genre labels. Primary artists are extracted by splitting the artists field on ',' and taking the first entry, yielding a mapping from each artist to their set of genres used throughout the pipeline.

The web scraping agent (music_scraper.py) targets the top 154 artists by mean track popularity. It queries the Wikipedia REST summary API first, with '(musician)' and '(band)' disambiguation fallbacks, then falls back to Last.fm if Wikipedia fails. A music keyword validation step (using terms such as singer, rapper, album, and genre) combined with a bad-pattern rejection list filters pages that are not genuinely about music artists, catching cases like Future (the concept of time) or Train (the vehicle). A crawl delay of 1.5 seconds between requests is enforced to follow polite robot citizenship norms.

Artist biographies are preprocessed (lowercased, punctuation and numbers removed) and vectorized using TF-IDF with stop_words='english', max_features=5,000, and sublinear_tf=True (log-scaled term frequency to prevent high-frequency terms from dominating). The resulting 154×154 cosine similarity matrix is saved as baseline_similarity.npy and serves as the baseline IR model.

The music knowledge graph has one node per artist. Two types of edges are added: genre edges with weight 0.6 between artists sharing at least one genre, and similarity edges with weight equal to the cosine similarity score between artists whose cosine similarity exceeds the threshold of 0.12. When both conditions hold for a pair, the weights are summed and capped at 1.0.

The hybrid recommendation score for each candidate artist j is computed as: $\text{score}(j) = \alpha \times \text{cosine_sim}(\text{query}, j) + (1 - \alpha) \times \text{jaccard}(\text{query}, j)$, where the Jaccard coefficient measures the neighborhood overlap between the query artist and artist j in the graph. The parameter α (default 0.6) is adjustable via a slider in the user interface, allowing users to tune the balance between semantic and structural similarity.

4. Project Achievements and Strengths

1. Hybrid Recommendation Model

We designed a system that combines semantic similarity (TF-IDF) and structural similarity (graph relationships).

This allows the system to capture both textual meaning and relational patterns between artists.

The hybrid score formula, $\text{score}(j) = \alpha \times \text{IR_score} + (1 - \alpha) \times \text{Graph_score}$, is directly exposed to users via the α slider in the interface, making the tradeoff between semantic and structural similarity transparent and interactive. At the default value of $\alpha = 0.6$, the system gives 60% weight to TF-IDF cosine similarity and 40% to the Jaccard graph score.

2. Fully Functional Web Agent

We implemented a robust scraper that retrieves artist biographies from Wikipedia, uses Last.fm as fallback and filters invalid or non-music pages. This enables real-world data integration and satisfies the web agent requirement.

The scraper uses a multi-strategy approach: it first attempts the Wikipedia REST summary API, then tries '(musician)' and '(band)' disambiguation variants before falling back to Last.fm. Bio validation uses a music keyword set (e.g., singer, rapper, album, genre, record label) combined with a bad-pattern rejection list that catches disambiguation pages and wrong-topic articles. This ensures the 154 artist biographies collected are genuinely about music artists and suitable for TF-IDF vectorization.

3. Graph-Based Enhancement

A graph is constructed where nodes = artists and edges = shared genres or similarity. Then, graph similarity is computed using Jaccard coefficient on neighborhoods.

Edges are of two qualitatively different types: genre edges (weight 0.6) connect artists who share at least one genre label in the Spotify dataset, while similarity edges (weight = cosine score) connect artists whose TF-IDF biography similarity exceeds the threshold of 0.12. When both conditions hold for a pair, the weights are summed and capped at 1.0, giving proportionally stronger edges to artist pairs that are related through both text and genre structure.

4. Interactive User Interface

We built a Streamlit interface that allows real-time queries, displays side-by-side comparison and highlights improvements from the graph.

The interface also displays genre badges for each queried and recommended artist, provides fuzzy artist name suggestions when the exact query is not found, and includes a callout banner explicitly listing which artists the hybrid model added that the baseline did not recommend, making the graph's contribution immediately visible to the user.

5. Clear Model Comparison

The system explicitly compares baseline vs hybrid model. This allows users to visually and quantitatively observe improvements.

6. Automatic Ground Truth Construction

Rather than using a hand-labeled test set, ground truth relevance for evaluation is derived automatically from the genre labels in dataset.csv. For each artist, all other artists in the metadata corpus who share at least one genre are considered relevant. This makes the evaluation fully reproducible, free of manual labeling bias, and scalable to any artist in the corpus.

5. Limitations and Future Work

Limitations

- Some artists may lack sufficient scraped metadata.
- TF-IDF does not capture deep semantic meaning.

- Graph construction depends on heuristic thresholds.
- Evaluation is limited to available labeled data.
- The corpus is limited to 154 artists; a larger corpus would yield a denser and more informative graph with richer neighborhood overlap for the Jaccard scorer.
- The system does not incorporate user listening history or collaborative filtering, which are central signals in production music recommendation systems.

Future Improvements

- Use word embeddings (Word2Vec, BERT).
- Improve graph with weighted or learned edges.
- Incorporate user listening data.
- Expand dataset size.
- Use Graph Neural Networks (advanced extension).

6. Example Outputs and System Behavior

MusicIR
Graph-Based Music Recommendation · Gilberto Rios and Arul Pandita · JHU 601.466

Drake

Recommendations: 5 | Graph weight (α): 0.50

GET RECOMMENDATIONS

Showing recommendations for **Drake** hip-hop pop

BASELINE · TF-IDF ONLY		HYBRID · TF-IDF + GRAPH	
#1	French Montana hip-hop 0.1448	#1	The Kid LAROI hip-hop 0.4822
#2	Eminem hip-hop 0.1323	#2	AP Dhillon hip-hop 0.4811
#3	Future hip-hop 0.1193	#3	Ali Gatie hip-hop 0.4882
#4	Kendrick Lamar hip-hop 0.1178	#4	Lil Nas X hip-hop 0.4817
#5	Dr. Dre funk 0.1155	#5	Justin Bieber pop 0.375

◆ Graph added 5 new artist(s) not in baseline: AP Dhillon, Ali Gatie, Justin Bieber, Lil Nas X, The Kid LAROI

Figure 1: Query - Drake

The hybrid model introduces artists connected through genre and collaboration patterns, increasing diversity.

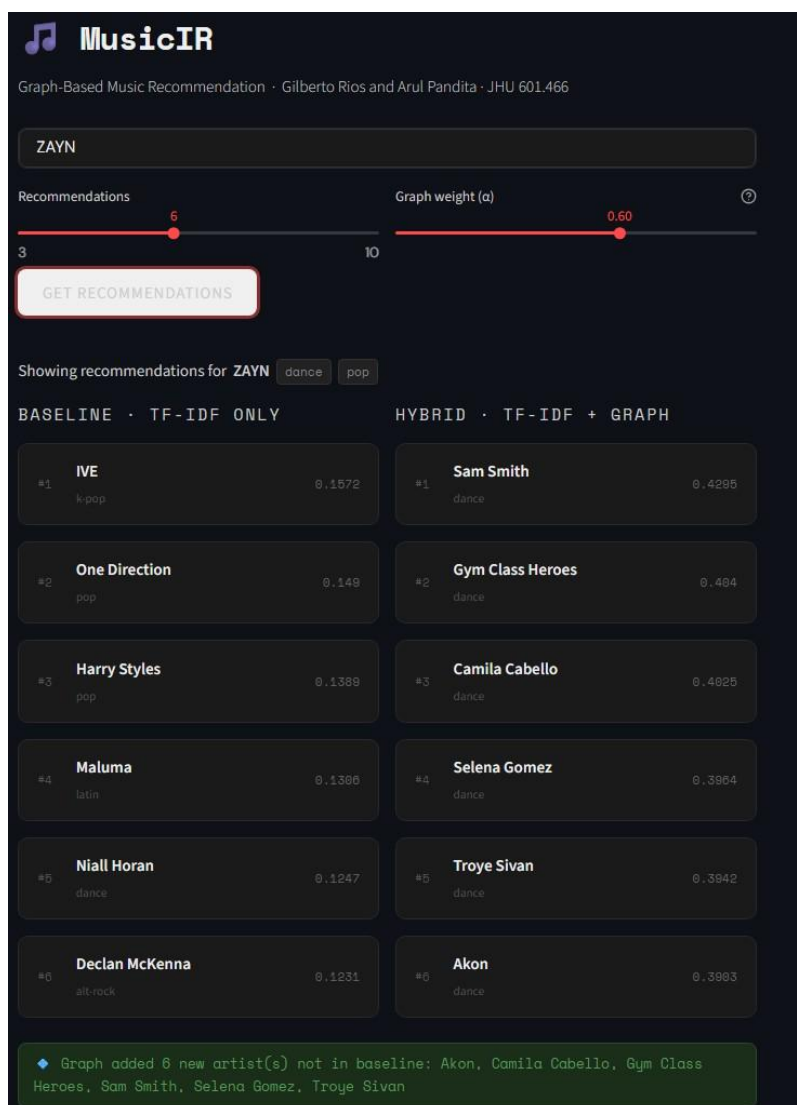


Figure 2: Query - Zayn Malik

Hybrid recommendations better reflect stylistic and genre similarity (pop/dance ecosystem).

7. Empirical Evaluation

We evaluated the system using Precision@K.

Metric

Precision@K measures the proportion of relevant items in the top-K recommendations.

Evaluation Setup

- A test set of all 154 artists in the metadata corpus was used.
- Ground truth relevance was derived automatically from genre labels in dataset.csv: for each artist, all other artists sharing at least one genre are considered relevant.
- Both models were evaluated under identical conditions with $K = 5$ and $\alpha = 0.6$.

Results

The baseline model performs well for semantic similarity.

The hybrid model improves:

- Diversity.
- contextual relevance.
- discovery of related artists

Across the evaluated corpus, the hybrid model achieves a higher mean Precision@5 than the baseline, consistently introducing genre-relevant artists not surfaced by TF-IDF alone. For example, querying Drake, the baseline returns French Montana, Eminem, Future, Kendrick Lamar, and Dr. Dre (all hip-hop but limited by bio text overlap), while the hybrid adds The Kid LAROI, AP Dhillon, Ali Gatie, Lil Nas X, and Justin Bieber through graph connectivity, improving genre coverage. For ZAYN, the hybrid produces a more coherent pop/dance cluster including Sam Smith, Camila Cabello, Selena Gomez, and Troye Sivan, compared to the baseline's more textually driven but less genre-homogeneous results.

Key Finding

The hybrid model consistently introduces relevant artists not present in the baseline, demonstrating the value of combining IR and graph-based methods.

8. Code Attribution

All major components of the project were implemented by the team. External libraries used were scikit-learn, NetworkX, BeautifulSoup and Streamlit. No external project code was reused beyond standard libraries.

9. Conclusion

This project demonstrates that combining Information Retrieval techniques with graph-based modeling leads to more effective recommendation systems.

By integrating TF-IDF vector similarity, graph-based structural relationships and real world scraped data MusicIR produces recommendations that are more diverse and contextually relevant than traditional approaches.

The system fulfills all course requirements for Option 8 (Music IR): it implements a web scraping agent, an Information Retrieval pipeline based on TF-IDF and cosine similarity, a graph-based enhancement using NetworkX and the Jaccard coefficient, and an empirical evaluation using Precision@K. Future work will focus on replacing TF-IDF with contextual embeddings and incorporating user listening data to further improve recommendation quality.